



SAPIENZA
UNIVERSITÀ DI ROMA

INGEGNERIA DELL'INFORMAZIONE, INFORMATICA E STATISTICA

Riconoscitore di falli nel calcio

AILAB: COMPUTER VISION AND NLP

Professore:

Pannone Daniele

Studenti e matricole:

Lorenzo Sabatino, 

Damiano Pronti, 

Guido Francesco Valente, 

Contents

1	Introduzione	1
2	Dataset	1
2.1	Creazione del DataSet	1
2.2	Preparazione del DataSet	2
2.3	DataSet splitting e formation	3
3	Dataloader	3
3.1	Creazione del DataLoader	3
4	Costruzione del modello	3
4.1	Addestramento	5
4.2	Allenamento iniziale	6
4.3	Hyperparameter Tuning	7
4.4	Training finale	7
5	Valutazione del modello	8
5.1	Caricamento del DataSet e del modello finale	8
5.2	Esecuzione dell'inferenza e metriche di valutazione	8
5.3	Conclusioni	9
	References	9

1 Introduzione

Il calcio è lo sport più popolare al mondo, e la sua crescente complessità rende sempre più importante l'impiego di tecnologie avanzate per supportare decisioni rapide e imparziali durante le partite. Tra gli aspetti più controversi vi è il riconoscimento dei falli: situazioni spesso soggette a interpretazioni soggettive, con un impatto rilevante sull'esito degli incontri. Negli ultimi anni, l'evoluzione dell'intelligenza artificiale e del deep learning ha aperto nuove possibilità nell'analisi automatica di immagini e video. Le reti neurali convoluzionali (CNN), in particolare, hanno mostrato ottimi risultati nella classificazione e nel riconoscimento di oggetti. In questo ambito, la rilevazione automatica dei falli rappresenta una sfida complessa, data la dinamicità delle riprese, la varietà di contatti fisici e la necessità di decisioni in tempo reale. Un sistema capace di identificare in modo affidabile i contrasti irregolari potrebbe supportare arbitri e VAR, riducendo errori e polemiche, e migliorando al contempo l'esperienza di spettatori e commentatori. Il nostro progetto si inserisce in questo contesto, con l'obiettivo di sviluppare un modello in grado di riconoscere automaticamente la presenza di un fallo a partire da un'immagine singola. Il sistema deve classificare la scena come "fallo" o "nessun fallo". A differenza di altri approcci che si concentrano su video in tempo reale tramite modelli complessi come YOLOv5/YOLOv8 o Faster R-CNN, il nostro lavoro ha adottato un modello più compatto, addestrato su immagini statiche. Questa scelta ha permesso di contenere tempi e risorse computazionali, pur ottenendo risultati promettenti in termini di accuratezza e capacità discriminativa.

Per comprendere meglio l'idea del modello e il suo funzionamento, è utile fare riferimento ad alcuni sistemi di intelligenza artificiale già esistenti con obiettivi analoghi. Tra questi troviamo *MatchVision*, un'IA in grado di riconoscere i falli, valutarne la gravità e generare commenti simili a quelli di un telecronista umano [RWJ⁺24]; l'*AI-Driven Image Recognition System*, che sfrutta tecniche di visione artificiale e un modello ibrido di deep learning per il rilevamento automatico di fuorigioco e falli [ZYY25]; e infine *AI-VAR* [Ans21], un sistema che utilizza una rete neurale convoluzionale personalizzata per classificare le immagini di gioco in contrasti regolari o falli.

2 Dataset

2.1 Creazione del DataSet

Poiché la rilevazione automatica dei falli nel calcio è un compito relativamente raro e complesso, non esiste un dataset pubblico standard specifico per questo scopo. Per questa ragione, abbiamo utilizzato un dataset reperito da Kaggle intitolato "**Football Tackles**" (zaikali/football-tackles) e lo abbiamo adattato ai nostri obiettivi. Il dataset è stato salvato e gestito all'interno di Google Drive per garantire accessibilità e persistenza durante l'uso in Google Colab.

Il dataset è suddiviso in due classi principali:

- Fouls – immagini contenenti situazioni di fallo,
- Clean_Tackles – immagini di tackle regolari, senza fallo.

Le immagini sono contenute nella cartella VAR del dataset e sono già etichettate in due directory distinte, corrispondenti alle due classi. Dopo un'analisi preliminare, abbiamo verificato che le immagini variano in dimensioni, risoluzioni e qualità: ciò ha richiesto una successiva fase di normalizzazione e preprocessing.



Figure 1: Campioni di esempio di immagini dalla classe var 400

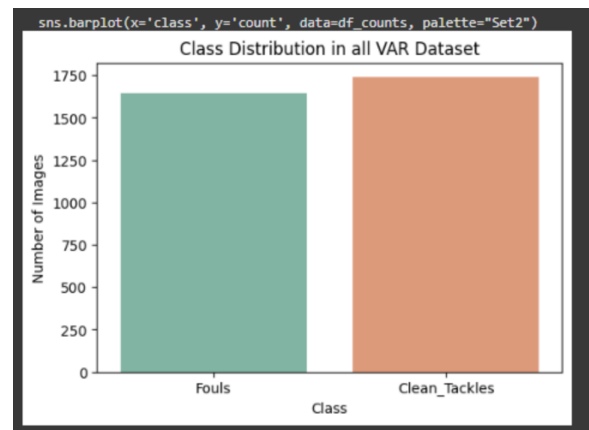


Figure 2: Distribuzione delle classi nell'intero dataset

2.2 Preparazione del DataSet

Per uniformare la risoluzione delle immagini, tutte sono state ridimensionate a (224, 224) pixel. Le immagini valide sono state poi normalizzate e convertite in tensori per l'addestramento del modello. Per aumentare la capacità generalizzativa del modello ed evitare fenomeni di overfitting, sono state applicate tecniche di data augmentation su tutte le immagini del training set. In particolare, le trasformazioni applicate includono:

- Flip orizzontale casuale
- Rotazione casuale
- Modifica casuale della luminosità
- Zoom e cropping casuale

Tutte le operazioni di trasformazione sono state implementate mediante la libreria `torchvision.transforms`.

2.3 DataSet splitting e formation

Dopo aver pulito e verificato il dataset, lo abbiamo suddiviso in tre sottoinsiemi secondo le seguenti proporzioni: Training set: 70% , Validation set: 15% e Test set: 15%

3 Dataloader

3.1 Creazione del DataLoader

Abbiamo sfruttato il meccanismo di **PyTorch** costruendo una pipeline basata sulle classi **Dataset** e **DataLoader** per caricare le immagini per il modello; per prima cosa, abbiamo organizzato le immagini in due cartelle (**no_fallo** e **fallo**) in modo da rendere immediato il riconoscimento delle etichette. Una funzione di utilità scorre ricorsivamente queste directory, raccogliendo i percorsi completi di tutte le immagini e assegnando a ciascuna un’etichetta binaria (0 per “nessun fallo”, 1 per “fallo”). Le due liste risultanti — una di percorsi e una di etichette — diventano la base su cui poggia la classe personalizzata che eredita da `torch.utils.data.Dataset`.

In fase di inizializzazione, questa classe riceve tali liste insieme a una serie di trasformazioni da applicare alle immagini; nel metodo `__len__` restituisce il numero totale di campioni, mentre in `__getitem__` si occupa di aprire il file, applicare le trasformazioni (come la normalizzazione secondo gli standard ImageNet) e restituire il tensore dell’immagine assieme alla sua etichetta.

Infine, ogni dataset è stato inserito in un **DataLoader** con *batch size* 32, ottimale per bilanciare stabilità e velocità di addestramento. Nel loader di training abbiamo abilitato lo **shuffle** per mescolare i dati a ogni epoca e ridurre l’overfitting, mentre nel loader di validazione lo shuffle è disattivato per garantire risultati ripetibili. Con **num_workers=2** il caricamento è stato parallelizzato su due processi, riducendo i tempi di I/O e mantenendo la GPU sempre alimentata.

Per verificare la correttezza di questo flusso, abbiamo implementato alcuni strumenti di controllo: da un lato, la visualizzazione di singole immagini con la loro etichetta per confermare che il *mapping* fosse corretto; dall’altro, l’estrazione e l’analisi di un intero batch per controllare le dimensioni dei tensori, range di valori e distribuzione delle classi. Grazie a queste verifiche, siamo sicuri che la pipeline di caricamento fornisca al modello dati accurati e uniformi.

4 Costruzione del modello

Per il riconoscimento automatico di falli nel calcio da immagini, è stata progettata una rete neurale convoluzionale (CNN) personalizzata, denominata **FoulClassifierCNN**. L’obiettivo del modello è classificare ciascuna immagine come contenente un fallo (fallo) o meno (no_fallo), ovvero un problema di classificazione binaria.

L'architettura del modello è stata definita interamente da zero, senza utilizzare modelli pre-addestrati. È composta da una serie di blocchi convoluzionali seguiti da livelli di pooling e fully connected, secondo lo schema:

Blocchi convoluzionali

La parte “feature extractor” di FoulClassifierCNN è articolata in quattro blocchi ripetuti, ciascuno composto da:

- **Conv2D + ReLU**

Un filtro 2D applica una convoluzione sull'immagine (o sulla precedente feature map), producendo un insieme di mappe di attivazione su cui la ReLU introduce non-linearità e favorisce la sparsity.

- **BatchNorm2D**

Questa normalizzazione dei batch stabilizza la distribuzione delle attivazioni, riduce il rischio di “covariate shift” interno e permette di usare learning rate più elevati.

- **MaxPool2D**

Riduce la dimensione spaziale delle feature map (ad esempio da $224 \times 224 \rightarrow 112 \times 112$, e così via) concentrando le informazioni più salienti in ogni regione, diminuendo al contempo il numero di parametri e il carico computazionale.

All'interno di ogni blocco, il numero di filtri aumenta progressivamente (ad esempio $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$), in modo da catturare, nei livelli più profondi, pattern sempre più complessi e ricchi di contesto (ad esempio la sovrapposizione di gambe, il contatto tra due giocatori, la postura corporea).

“Head” completamente connesso

Dopo l'ultimo MaxPool, le feature map vengono appiattite e passate a uno stack di layer fully connected:

- **Linear + ReLU**

Un primo livello lineare riduce le migliaia di valori provenienti dalle feature map in un vettore di dimensione intermedia. La ReLU successiva mantiene la capacità di rappresentare relazioni non-linearmente separabili.

- **Dropout**

Con un tasso studiato sperimentalmente (ad esempio 0.5), il dropout “spenge” in modo casuale parte dei neuroni durante il training, agendo da regolarizzatore e riducendo l'overfitting.

- **Linear (2 unità)**

L'ultimo layer lineare proietta il vettore su due logit, uno per ciascuna classe (“fallo” vs “no_fallo”).

Output e interpretazione

I due logit prodotti vengono convertiti in probabilità tramite softmax, consentendo di interpretare in modo diretto quanto il modello “creda” che nell’immagine sia presente un fallo. In fase di inferenza, basta confrontare le due probabilità per assegnare la classe più probabile.

4.1 Addestramento

Una volta definita l’architettura del modello, è stata avviata la fase di addestramento, suddivisa in tre momenti principali:

1. Un primo addestramento del modello
2. Fase di tuning e validazione, per identificare la miglior combinazione di iperparametri.
3. Fase di training finale, eseguita sull’intero dataset (training + validation) con la configurazione ottimale.

La fase di addestramento è stata progettata per essere robusta, tracciabile e facilmente ottimizzabile. L’obiettivo era non solo migliorare le prestazioni del modello, ma anche monitorare in tempo reale l’andamento delle metriche principali per prevenire fenomeni come overfitting o underfitting. Per farlo, abbiamo integrato vari componenti fondamentali nel training loop.

Il processo ha avuto inizio con l’inizializzazione degli strumenti essenziali:

- Il modello `FoulClassifierCNN` è stato istanziato e spostato sul dispositivo disponibile (GPU o CPU).
- La funzione di perdita utilizzata è stata la **CrossEntropyLoss**, standard per compiti di classificazione.
- L’**ottimizzatore** scelto è Adam, noto per la sua stabilità e adattabilità, impostato con un learning rate iniziale di $1e-4$.
- È stato aggiunto uno **scheduler CosineAnnealingLR** per modificare il learning rate in modo dinamico durante le epoche.
- È stato configurato un **early stopping** con una soglia di pazienza pari a 10 epoche consecutive senza miglioramento della validation loss.

È stato inizializzato un `SummaryWriter` per il **monitoraggio su TensorBoard**.

4.2 Allenamento iniziale

Durante ogni epoca, nella fase di addestramento iniziale, il modello ha alternato le seguenti fasi:

1. Fase di addestramento:

- Il modello veniva messo in modalità `train()`.
- Per ogni batch di immagini:
 - I tensori venivano spostati sul dispositivo.
 - Si calcolavano gli output tramite `forward pass`.
 - Veniva calcolata la `loss` e si eseguiva il `backpropagation`.
 - L'ottimizzatore aggiornava i pesi.
- Alla fine della fase, venivano calcolati:
 - La **loss media** su tutto il training set.
 - L'**accuratezza** totale (numero di predizioni corrette / totale delle predizioni).

2. Fase di validazione:

- Il modello veniva valutato in modalità `eval()` senza calcolo dei gradienti.
- Si calcolavano `loss` e `accuratezza` su `validation set`.

3. Logging su TensorBoard:

- `Loss` e `accuracy`, sia per il training che per la validazione, venivano registrate a ogni epoca.
- Questo ha permesso di visualizzare in tempo reale le metriche su TensorBoard.

4. Gestione dell'early stopping:

- Se la `validation loss` migliorava, il modello veniva salvato (`best_model.pth`).
- Altrimenti, veniva incrementato un contatore. Se il contatore raggiungeva 10, il training veniva interrotto.

Grazie a questa struttura, è stato possibile monitorare in modo dettagliato l'intero processo di addestramento, evitare l'overfitting e selezionare automaticamente il modello con le migliori prestazioni su `validation set`. L'uso di TensorBoard si è rivelato particolarmente utile per visualizzare graficamente le tendenze delle metriche durante l'addestramento, fornendo insight preziosi per eventuali modifiche.

4.3 Hyperparameter Tuning

L'Hyperparameter Tuning è il processo mediante il quale si identifica la migliore combinazione di configurazioni di training al fine di massimizzare le prestazioni del modello. Tipicamente, questo processo include la selezione e l'ottimizzazione di parametri quali:

- Tasso di apprendimento (`lr`)
- Dimensione del batch
- Tipo di ottimizzatore (Adam, SGD, ecc.)
- Numero di filtri in ogni blocco convoluzionale
- Tasso di abbandono
- Tipo di scheduler

Il nostro processo utilizza Optuna, una libreria di ottimizzazione bayesiana, per il *tuning* automatico degli iperparametri di un modello di classificazione, con l'obiettivo di massimizzare l'accuratezza sul validation set. L'ottimizzazione prevede due trial, durante i quali si ottimizzano tre parametri chiave: il tasso di apprendimento (`lr`), su scala logaritmica tra $1e-5$ e $1e-2$; la dimensione del batch (`batch_size`), scelta tra 32 e 64; il dropout (`dropout`), in un intervallo continuo tra 0.2 e 0.5.

Per ogni trial, si definisce la struttura del modello con i parametri suggeriti, si crea l'ottimizzatore Adam e uno scheduler StepLR che riduce gradualmente il learning rate per una convergenza stabile. Il modello viene allenato per 10 epoche con il processo descritto precedentemente e poi valutato. Alla fine, viene restituita l'accuratezza come metrica da massimizzare.

Dopo i due trial, si sceglie la configurazione migliore e si esegue un retraining di 10 epoche con questi iperparametri ottimali. Il modello finale viene salvato in `best_model_optuna.pth`.

4.4 Training finale

Dopo la fase di tuning e validazione viene fatto un training finale sull'intero dataset (training + validation) con la configurazione ottimale. Inizialmente vengono unificati i dataset, `train_dataset` e `val_dataset` vengono combinati in un unico `ConcatDataset` che sarà usato per l'addestramento finale. Di seguito si ricostruisce il modello finale per includere il valore di dropout ottimale. Poi si configurano l'ottimizzatore, con il learning rate ottimale, lo scheduler e il criterio di perdita. Dopo di che si avvia l'addestramento del modello sull'intero dataset per 10 epoche.

Infine si salva il modello nel file `best_model_optuna.pth`.

5 Valutazione del modello

5.1 Caricamento del DataSet e del modello finale

È stato inizializzato un DataLoader per il test set, utilizzando le stesse trasformazioni applicate in fase di validazione. Questo garantisce coerenza nel pre-processing e rende il confronto delle prestazioni più accurato:

- I percorsi e le etichette sono stati letti tramite `load_image_paths_and_labels`.
- I dati sono stati caricati in batch (batch size 32) e senza shuffle, in quanto l'ordine dei campioni non influisce sul testing.

Il modello migliore ottenuto durante l'addestramento (quello con la validation loss più bassa) viene caricato dalla memoria locale o da Google Drive, a seconda di cosa scegliamo. È stata gestita la compatibilità con CUDA per sfruttare eventuali GPU disponibili, e il modello è stato impostato in modalità `eval()` per disattivare il dropout e batch normalization.

5.2 Esecuzione dell'inferenza e metriche di valutazione

Per ottimizzare le prestazioni e risparmiare memoria, è stato utilizzato `torch.no_grad()`. Durante questa fase:

- Le immagini del test set sono state processate in batch.
- Il modello ha prodotto predizioni che sono state confrontate con le etichette reali.
- Le predizioni (`all_preds`) e le etichette (`all_labels`) sono state memorizzate per il calcolo delle metriche.

Sono state utilizzate metriche classiche per compiti di classificazione binaria:

- **Accuracy**: percentuale di predizioni corrette.
- **Precision, Recall, F1-score**: metriche derivate dal bilancio tra falsi positivi e falsi negativi.
- È stato inoltre stampato il **classification report** di sklearn, che fornisce una panoramica dettagliata per ciascuna classe ("fallo" e "no_fallo").

Come ultimo passo è stata stampata una **confusion matrix** con seaborn, che aiuta a valutare visivamente gli errori del modello, rendendo più chiare le aree dove il modello effettua valutazioni con più o meno precisione.

Classification Report:				
	precision	recall	f1-score	support
no_fallo	0.77	0.66	0.71	435
fallo	0.69	0.79	0.73	412
accuracy			0.72	847
macro avg	0.73	0.72	0.72	847
weighted avg	0.73	0.72	0.72	847

Figure 3: Report del modello

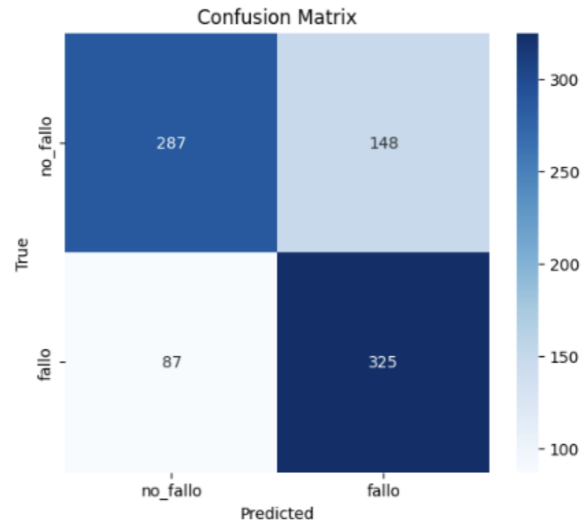


Figure 4: Matrice di confusione

5.3 Conclusioni

Il nostro risultato finale deriva anche da alcuni problemi riscontrati nell'allenamento, uno fra tutti quello dell'overfitting; nonostante il modello avesse raggiunto un'accuratezza dell'80% in fase di training, durante i test non superava il 60%. Per ovviare a questo problema, abbiamo diminuito i trial nella fase di Hyperparameter Tuning e l'aumento del numero di immagini. Questa soluzione ha richiesto tempo, poiché è stato necessario testare numerosi valori per il trial prima di individuare una configurazione efficace. Nonostante ciò il modello comunque non è molto preciso, soprattutto quando si parla delle immagini che non sono falli, infatti ha un accuracy più o meno del 70%. Nonostante ciò crediamo che con un database più ampio e con una scelta migliore delle immagini per effettuare training e test si possa aumentare di molto la precisione del modello.

References

- [Ans21] Aamir Ahmad Ansari. Var-cnn: Foul or clean tackle detection in football using cnn. <https://aamir07.medium.com/var-cnn-football-foul-or-clean-tackle-4ff6629c83db>, 2021.
- [RWJ⁺24] Jiayuan Rao, Haoning Wu, Hao Jiang, Ya Zhang, Yanfeng Wang, and Weidi Xie. MatchVision: Artificial Intelligence in Football Refereeing. <https://jyrao.github.io/UniSoccer/>, 2024.
- [ZYY25] Qianwei Zhang, Lirong Yu, and WenKe Yan. Ai-driven image recognition system for automated offside and foul detection in football matches using computer vision. *International Journal of Advanced Computer Science and Applications*, 16(1), 2025. Disponibile a: <http://dx.doi.org/10.14569/IJACSA.2025.01601114>.